Regime-Aware Machine Learning for Equity Trading

Technical Pattern Detection, Triple-Barrier Labeling, and Ensemble Classification on SPY Daily Data

Zeineb Turki

April 2026

A thesis-style project report
prepared for academic supervision review

Table of Contents

1	Abstract
2	Introduction
3	Literature and Conceptual Background
4	Project Architecture
5	Data Sources
6	Pattern Detection Methodology
7	Trendline Touch Analysis
8	Triple-Barrier Labeling
9	Feature Engineering
10	Model Methodology
11	Experiments and Results
12	Discussion
13	Limitations
14	Future Work
15	Conclusion
16	References
A	Appendix A: Detector Parameters
B	Appendix B: Complete Feature List

1. Abstract

This project develops a regime-aware machine learning pipeline for equity trading, applied to 4,023 daily bars of SPDR S&P; 500 ETF Trust (SPY) data spanning 2010-01-04 to 2025-12-30. The system detects technical price patterns — support/resistance levels, triangle formations, price channels, and multiple tops/bottoms — using pivot-based and chunk-based algorithms with linear regression trendline fitting. Each detected event is labeled via the triple-barrier method (Lopez de Prado, 2018), producing 221 labeled events with three classes: long (88), short (64), and no_trade (69). A feature engineering module computes 55 features per event, combining technical indicators, pattern geometry, and touch-counting statistics. Random Forest and Bagging classifiers are trained using a strict temporal split to avoid lookahead bias. The Random Forest achieves 42.2% test accuracy and 0.347 macro-F1, outperforming a stratified baseline (37.8% accuracy). Key discriminative features include recent returns, moving average distances, and volume statistics. The project demonstrates a transparent, modular, and reproducible approach to ML-based trading signal classification.

Keywords: technical analysis, pattern detection, triple-barrier labeling, random forest, bagging, ensemble methods, event-driven trading, feature engineering, temporal cross-validation.

2. Introduction

2.1 Motivation

Financial markets exhibit complex, non-stationary dynamics that challenge traditional trading strategies. Technical analysis — the study of price action and chart patterns — has been practiced for decades, yet its systematic quantification and integration with machine learning remains an active research area. This project addresses the question: can algorithmically detected price patterns, combined with modern ML classifiers, produce actionable trading signals?

The core challenge is threefold. First, pattern detection must be rigorous: naively fitting trendlines to noisy price data produces excessive false positives. Second, event labeling must avoid the hindsight bias common in technical analysis — the triple-barrier method provides a principled forward-looking label. Third, model evaluation must respect temporal ordering to avoid information leakage.

2.2 Objectives

This project pursues the following objectives:

• Implement robust, parameter-tuned pattern detectors for triangles, channels, support/resistance, and multiple tops/bottoms.

• Develop a trendline touch-counting methodology that quantifies how well price action respects detected boundaries.

• Apply triple-barrier labeling to generate event-level ML targets.

• Engineer a comprehensive feature set combining technical indicators with pattern geometry and touch statistics.

• Train and compare ensemble classifiers (Random Forest and Bagging) using strict temporal splitting.

• Provide a transparent, reproducible pipeline with visible statistics at every stage.

2.3 Scope

The analysis focuses on SPY daily data (2010-01-04 to 2025-12-30, 4,023 trading days). While the methodology is general, this project does not implement live trading, transaction cost modeling, or portfolio optimization. The emphasis is on detection quality, labeling correctness, and classification transparency.

3. Literature and Conceptual Background

3.1 Technical Pattern Detection

Lo, Mamaysky, and Wang (2000) provided the first rigorous statistical framework for evaluating technical patterns. They showed that certain patterns (head-and-shoulders, double tops, triangles) carry incremental information beyond random walk models. Their kernel-regression approach to detecting smooth extrema inspired the swing-pivot detection used in this project.

Bulkowski (2005, 2021) catalogued hundreds of chart patterns with empirical success rates. His classification of triangles into ascending, descending, and symmetric subtypes, along with breakout statistics, informs our subtype classification logic.

3.2 Triple-Barrier Labeling

Lopez de Prado (2018) introduced the triple-barrier method in *Advances in Financial Machine Learning*, Chapter 3. Unlike fixed-horizon returns, the triple barrier sets a profit-target barrier (upper), stop-loss barrier (lower), and a time-expiry barrier. The label is determined by which barrier is breached first, producing a natural three-class classification: long, short, and no_trade. This avoids the arbitrary discretization of returns into bins and provides labels that are interpretable as trading decisions.

3.3 Ensemble Methods

Random Forest (Breiman, 2001) constructs an ensemble of decision trees, each trained on a bootstrap sample with a random subset of features considered at each split. This double randomization reduces variance while maintaining low bias. Bagging (Bootstrap Aggregating; Breiman, 1996) uses bootstrap sampling alone, without feature subsetting, resulting in higher correlation between trees but sometimes competitive performance when individual features are highly informative.

Both methods are well-suited to financial classification: they handle mixed feature types, are robust to outliers, and provide natural feature importance rankings. Their ensemble structure also reduces the risk of overfitting to any single decision boundary.

3.4 Risk Management and Labeling

Traditional technical analysis provides entry signals but lacks a systematic framework for exit management. The triple-barrier approach (de Prado, 2018) addresses this by defining three exit scenarios: profit target, stop loss, and time expiry. This framework converts the continuous price series into discrete events with well-defined outcomes, enabling supervised classification.

The choice of ATR-based barrier sizing (rather than fixed percentage thresholds) ensures that barriers adapt to current market volatility. During high-volatility periods, barriers widen to accommodate larger price swings, while in calm markets, barriers tighten to capture smaller moves. This adaptive behavior is critical for a dataset spanning 16 years, during which SPY's daily ATR varied by an order of magnitude.

3.5 Feature Engineering for Financial ML

Effective feature engineering is critical for financial ML (de Prado, 2018). Technical indicators such as RSI, MACD, Bollinger Bands, and moving average spreads encode momentum, mean-reversion, and volatility regimes. Combining these with event-specific features (pattern geometry, touch counts, containment ratios) creates a richer representation that captures both market context and pattern quality.

4. Project Architecture

The project follows a modular pipeline design with clear separation between data, detection, labeling, features, and modeling. Each module is independently testable and the pipeline flows unidirectionally from raw data to predictions.

Module	Path	Purpose
Data Loader	src/data/load_data.py	Load SPY from CSV, yfinance, or Alpha Vantage
ATR Utility	src/data/utils.py	Average True Range computation
S/R Detector	src/patterns/support_resistance.py	Rolling support/resistance with stability filter
Triangle Detector	src/patterns/triangles.py	Pivot + linregress triangle detection
Channel Detector	src/patterns/channels.py	Chunk-based channel with dynamic window
Multi Top/Bottom	src/patterns/multiple_tops_bottoms.py	Rolling extremes + trend confirmation
Pivot Utilities	src/patterns/pivots.py	Swing detection, containment, touch counting
Scanner	src/patterns/scanner.py	Orchestrator for all 4 detectors
Labeling	src/labeling/label_events.py	Triple-barrier event labeling
Indicators	src/features/indicators.py	32 technical indicators
Feature Builder	src/features/build_features.py	Event-based feature matrix construction
ML Training	src/models/train.py	RF, Bagging, Baseline training + evaluation

4.2 Data Flow

The data flows through the pipeline in a strict unidirectional fashion:

1. Data Loading — Raw OHLCV data is loaded from CSV (or downloaded from yfinance/Alpha Vantage). The data is cleaned, sorted, and validated.
2. Indicator Computation — 32 technical indicators are computed at each bar using only backward-looking windows.
3. Pattern Detection — Four detectors scan the data independently, each producing a pattern column and optional metadata.
4. Event Flagging — The scanner combines all detector outputs into a unified has_event flag.
5. Labeling — Each flagged event is labeled using the triple-barrier method.
6. Feature Assembly — Bar-level indicators are joined with pattern geometry features at each event timestamp.
7. Model Training — The feature matrix and labels are split temporally and fed to ensemble classifiers.

4.3 Notebooks

Notebooks provide the interactive analysis layer. The project includes the following notebooks, each focused on a specific analysis task:

Notebook	Purpose
03_pattern_detection_progress_report	Detector tuning and before/after analysis

Notebook	Purpose
03_pattern_validation	Event quality metrics and forward-return analysis
04_pattern_structure_validation	Triangle detection gallery with pivot charts
05_triple_barrier_labeling	Labeling pipeline and sensitivity analysis
06_channel_gallery	Channel detection gallery
07_data_source_comparison	yfinance vs Alpha Vantage comparison
08_detector_touch_analysis	Trendline touch counting validation
09_feature_engineering	Feature computation and analysis
10_model_training	RF and Bagging training with full evaluation
11_experiment_summary	Consolidated results and statistics

5. Data Sources

5.1 Primary Source: Yahoo Finance via yfinance

The primary dataset consists of 4,023 daily OHLCV bars for SPY, downloaded from Yahoo Finance using the yfinance Python library. The data spans 2010-01-04 to 2025-12-30, covering approximately 16 years of trading activity. Price ranges from $77.36 to $690.38, with an annualized return of 14.0%.

Statistic	Value
Rows	4,023
Date range	2010-01-04 to 2025-12-30
Price range	$77.36 - $690.38
Annualized return	14.0%
Average daily ATR(14)	$3.37
Columns	Open, High, Low, Close, Volume

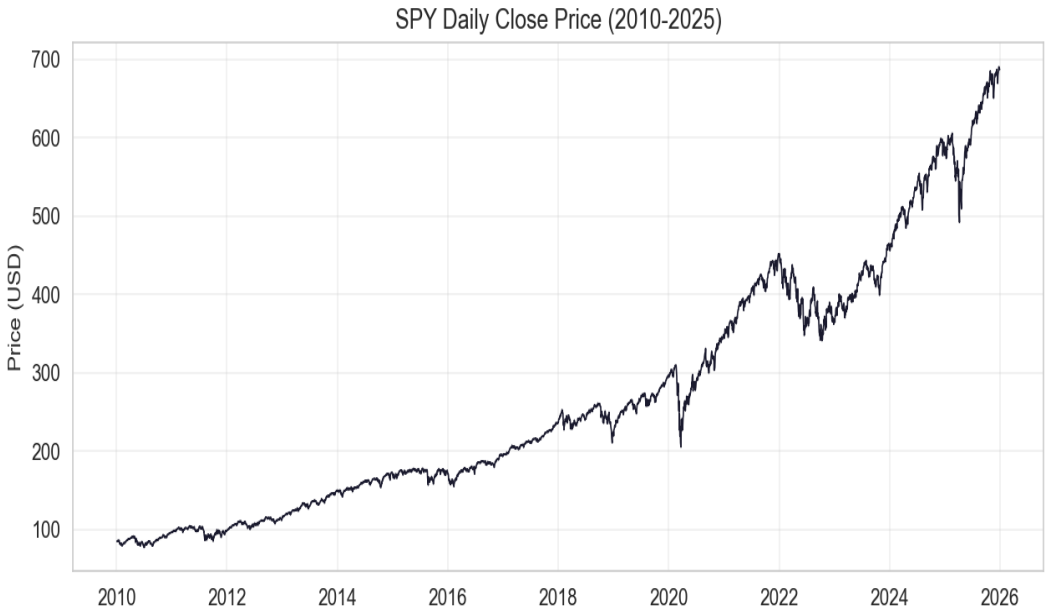

Figure 1: SPY daily close price, 2010-2025.

5.2 Data Exploration

Before pattern detection, it is important to understand the statistical properties of the raw data. The SPY daily returns exhibit the well-known features of financial time series: approximate symmetry around zero, leptokurtosis (fat tails), and volatility clustering.

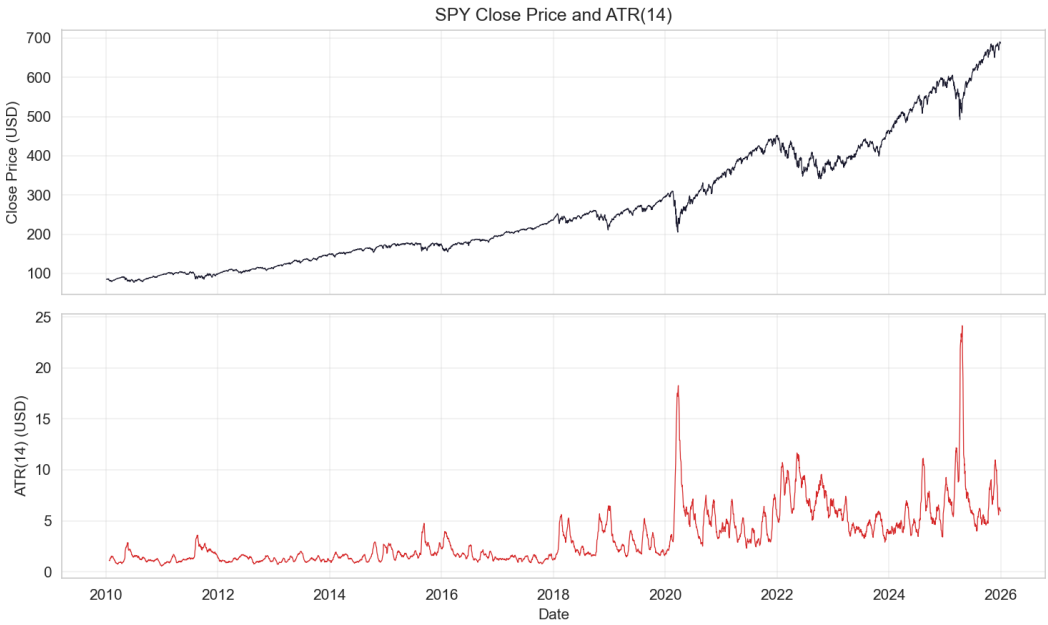

Figure 2: SPY close price (top) and ATR(14) (bottom). Note the volatility spikes during the 2011 debt crisis, 2018 volatility event, 2020 COVID crash, and 2022 bear market.

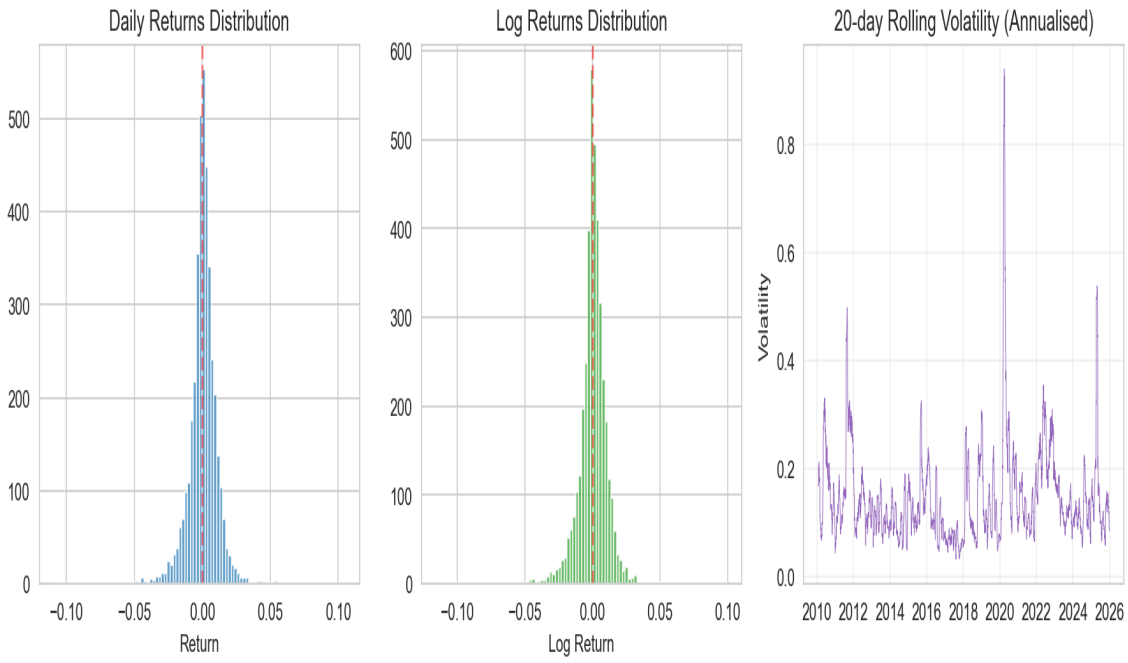

Figure 3: Daily returns (left), log returns (center), and 20-day rolling annualised volatility (right).

Daily return statistics: mean=0.0006, std=0.0108, min=-0.1094, max=0.1050, skew=-0.335, kurtosis=12.118. The negative skew and excess kurtosis confirm the non-Gaussian nature of returns, justifying the use of ATR-based (rather than standard-deviation-based) thresholds throughout the detection pipeline.

5.3 Secondary Source: Alpha Vantage

The system supports Alpha Vantage as a secondary data source for robustness testing. The Alpha Vantage loader downloads daily adjusted data via their REST API (CSV format) and normalizes it to the same OHLCV schema. A comparison utility measures date coverage, price differences, volume correlation, and detector behavior differences between sources. When the API key is not available, the comparison framework can be tested using a simulated second source (small random perturbation of the primary data).

6. Pattern Detection Methodology

Four pattern detectors scan the SPY data and flag 221 events (5.5% of bars). Each detector uses ATR-adaptive thresholds, stability filters, and cooldown periods to control false positives.

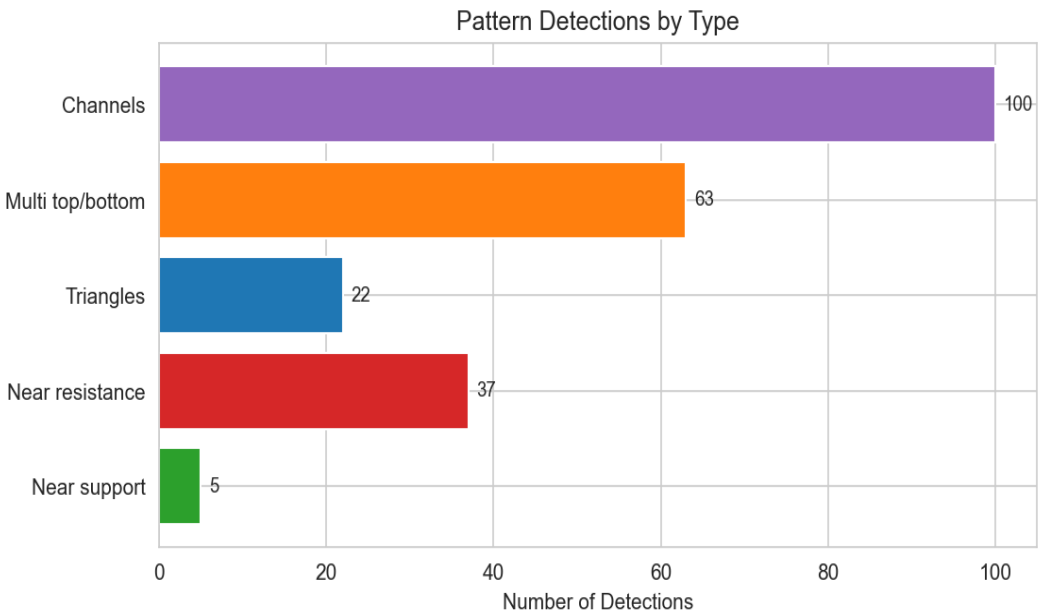

Figure 2: Number of detections by pattern type.

Detector	Count	% of Bars
Near support	5	0.12%
Near resistance	37	0.92%
Triangles	22	0.55%
Multi top/bottom	63	1.57%
Channels	100	2.49%
Total events	221	5.5%

6.1 Support and Resistance

The support/resistance detector computes rolling minimum of Lows (support) and rolling maximum of Highs (resistance) over a 50-bar window. A signal fires when the current Close is within 0.3 x ATR(14) of either level. Two filters reduce false positives: (1) a stability filter requires the level to be unchanged for 5 bars, preventing signals during trending markets; (2) a 10-bar cooldown suppresses repeated signals from the same approach.

6.2 Triangle Detection

Triangles are detected using a pivot-point approach inspired by Lo et al. (2000). Within a 25-bar lookback window, swing highs and lows are identified using a +/-3 bar neighborhood. Linear regression (scipy.stats.linregress) is fitted to the swing highs and lows separately, requiring |r| >=

0.85 on each trendline. Intercepts are adjusted so the upper line caps all swing highs and the lower line floors all swing lows, creating the classic bounding-line geometry.

The triangle is classified by slope signs: ascending (flat upper, rising lower), descending (falling upper, flat lower), or symmetric (both converging). Flatness is defined as |slope| < 0.1 x ATR / window. A signal fires on breakout (High or Low exceeds recent range by 0.3 x ATR) or on an upper-test for descending triangles.

6.3 Channel Detection

Channels are detected using the chunk-based method from the TrendLineChannelDetection reference notebook. The lookback window is dynamically optimized: 31 windows (25-55 bars) are tested, and the one producing the tightest channel at the current bar is selected. Each window is divided into 5-bar chunks; the max-High and min-Low from each chunk are used as representative points for polyfit(degree=1).

Validation requires: parallel slopes (relative difference < 25%, same sign), width between 1-6 x ATR, and containment >= 60%. A signal fires when the current Close is within 0.3 x ATR of either boundary.

6.4 Multiple Tops and Bottoms

Multiple tops are detected when the rolling maximum of Highs (50-bar window) stays elevated while a 5-bar close-trend slope is negative. Multiple bottoms are the mirror: rolling minimum stays depressed with positive close slope. A 10-bar cooldown prevents signal clustering.

6.5 Detection Overview

Across all four detectors, the system identifies 221 event bars out of 4,023 total bars, for a combined event rate of 5.5%. This is within the target range of 3-8% — high enough to produce a meaningful training set, but low enough to avoid diluting signal quality with noise.

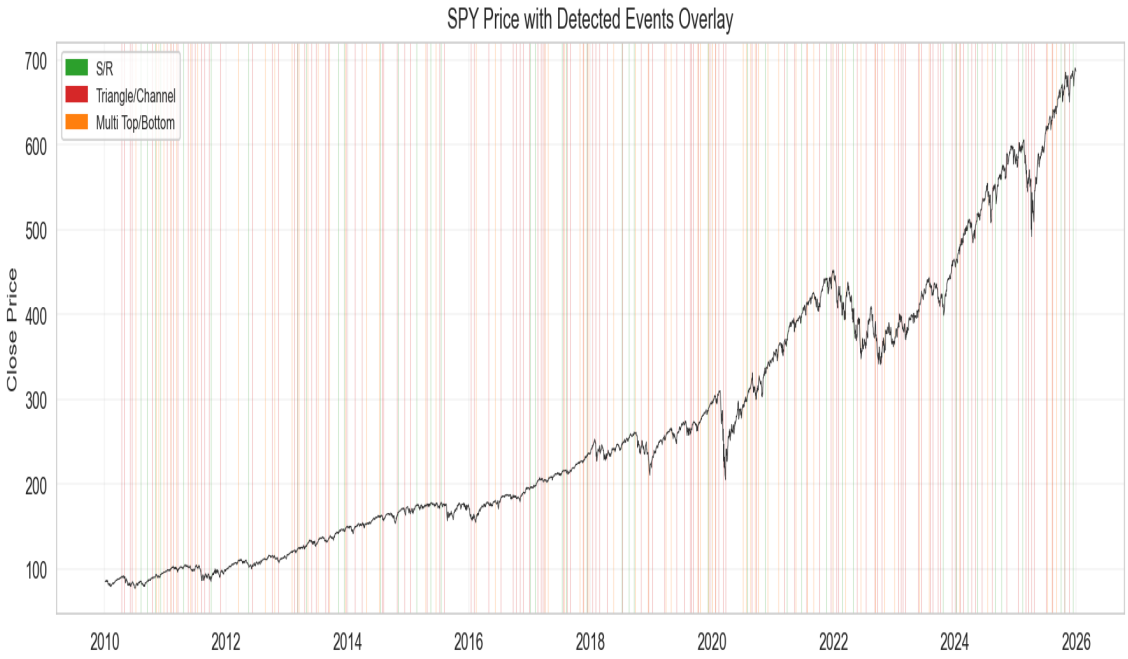

Figure 5: SPY price with all detected events overlaid. Green = S/R, red = triangle/channel, orange = multi top/bottom.

The event timeline shows that detections are distributed across the full 16-year period, with higher density during volatile markets (2011, 2018-2020, 2022). This is expected: volatile periods produce more price reversals and boundary tests. The cooldown mechanism prevents excessive clustering within any single volatile episode.

Detections by Year

Year	Events	Bars	Event Rate
2010	13	252	5.2%
2011	17	252	6.7%
2012	7	250	2.8%
2013	18	252	7.1%
2014	10	252	4.0%
2015	9	252	3.6%
2016	11	252	4.4%
2017	20	251	8.0%
2018	13	251	5.2%
2019	15	252	6.0%
2020	16	253	6.3%
2021	13	252	5.2%
2022	13	251	5.2%
2023	15	250	6.0%

Year	Events	Bars	Event Rate
2024	15	252	6.0%
2025	16	249	6.4%

Table: Annual detection counts and event rates.

7. Trendline Touch Analysis

A key contribution of this project is the systematic counting and analysis of trendline touches. For each triangle and channel detection, the system counts how many bars have their High (for upper lines) or Low (for lower lines) within a tolerance of 0.3 x ATR of the fitted trendline.

7.1 Touch Counting Methodology

A touch is registered when |price_extreme - trendline_value| <= tolerance. For upper lines, we measure the distance from each bar's High to the trendline; for lower lines, from each bar's Low. A violation is counted when a bar breaches the line beyond tolerance. The touch tolerance is set to 0.3 x ATR(14) at the event bar, providing an adaptive threshold that accounts for current volatility.

7.2 Triangle Touch Statistics

Metric	Value
Number of triangles	22
Avg total touches	10.1
Avg containment	0.860
Min containment	0.720
Avg upper touches	5.7
Avg lower touches	4.4

7.3 Channel Touch Statistics

Metric	Value
Number of channels	100
Avg total touches	7.6
Avg containment	0.984
Min containment	0.840
Avg upper touches	4.8
Avg lower touches	2.8
Avg channel width (ATR)	3.42

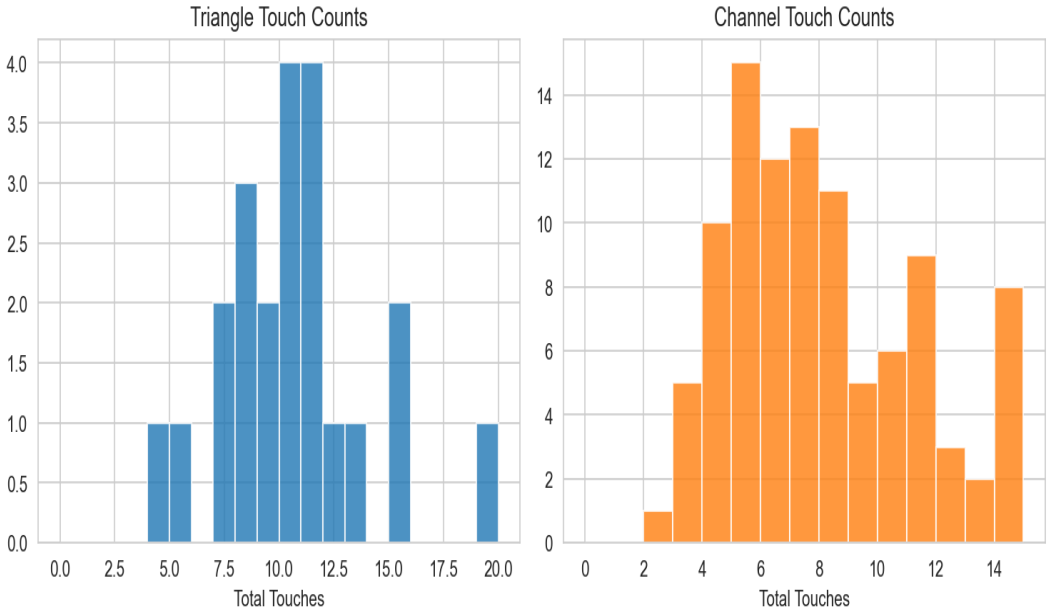

Figure 3: Distribution of total trendline touches for triangles and channels.

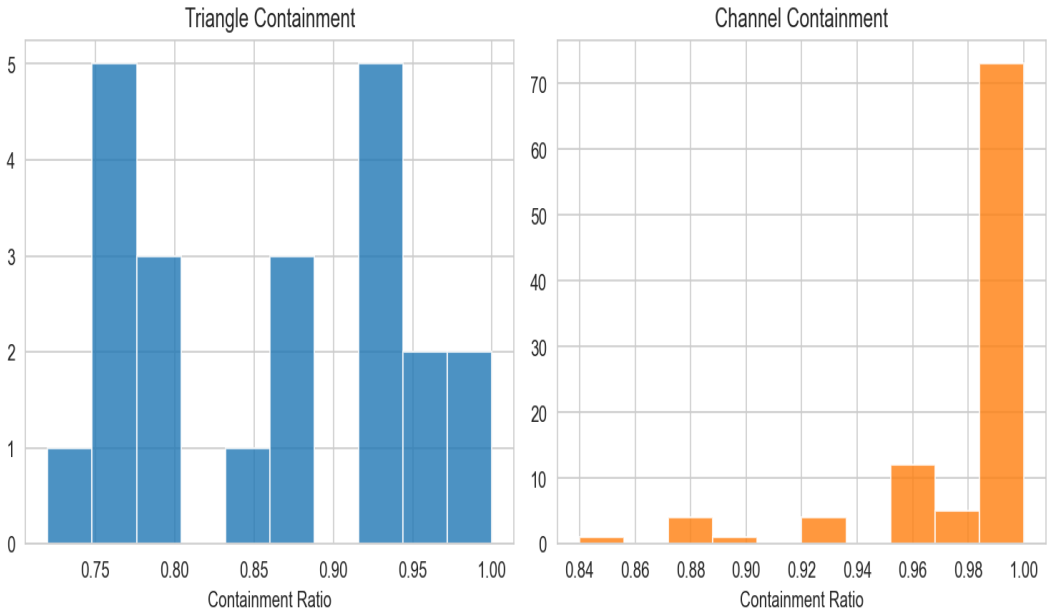

Figure 7: Distribution of containment ratios for triangles and channels.

7.4 Channel Width Analysis

The channel width, measured in ATR units, ranges from 1.0 (the minimum allowed) to 6.0 (the maximum allowed), with a mean of 3.42 ATR. Narrower channels indicate tighter price consolidation, while wider channels represent broader trend movements.

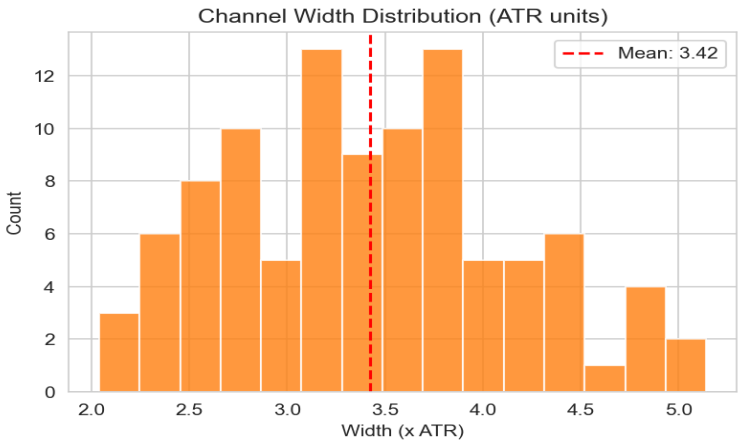

Figure 8: Distribution of channel widths in ATR units.

7.5 Touch Quality Assessment

The mean touch error (distance from price extreme to trendline at touch points) provides a quality metric for each detection. Lower errors indicate closer adherence to the fitted trendline.

For triangles, the average upper-line touch error is 0.265 USD, and the average lower-line touch error is 0.220 USD. For channels, the corresponding values are 0.467 USD and 0.267 USD.

The touch errors scale with price level and volatility — errors are larger for recent detections (when SPY trades above $500) compared to early detections (SPY near $100). Normalizing by ATR would make these errors comparable across different price regimes, and this normalization is already embedded in the tolerance threshold (0.3 x ATR).

8. Triple-Barrier Labeling

Following Lopez de Prado (2018), each detected event is labeled by walking forward from the event bar and checking which of three barriers is breached first:

• **Upper barrier** (profit target): entry_price + 2.0 x ATR(14)
• **Lower barrier** (stop loss): entry_price - 2.0 x ATR(14)
• **Time barrier**: 10 bars after entry

The labeling produces 221 events: 88 long (39.8%), 64 short (29.0%), and 69 no_trade (31.2%).

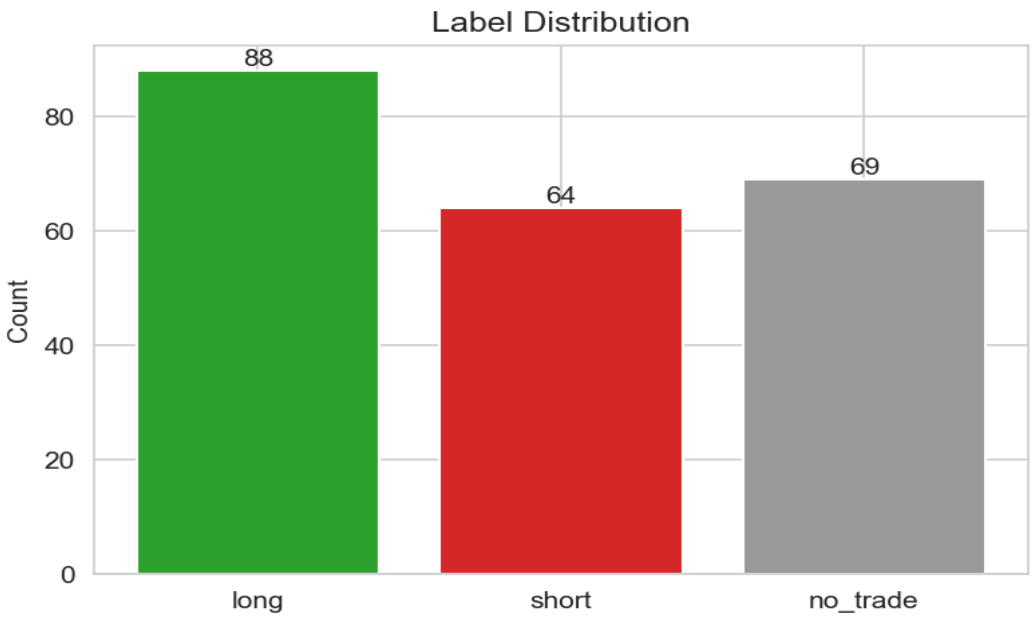

Figure 9: Label distribution across all events.

8.1 Barrier Mechanics

The triple-barrier method works as follows: for each event bar, the entry price is the Close of that bar. The upper barrier is set at entry + pt_mult x ATR(14), and the lower barrier at entry - sl_mult x ATR(14). The algorithm walks forward through subsequent bars, checking whether the bar's High breaches the upper barrier or its Low breaches the lower barrier. If neither barrier is breached within max_holding bars, the time barrier triggers and the event is labeled no_trade.

When both barriers are breached on the same bar (a wide-range day), the Close is used as a tiebreaker: if Close >= entry, the label is long; otherwise short.

8.2 Return and Holding Period Analysis

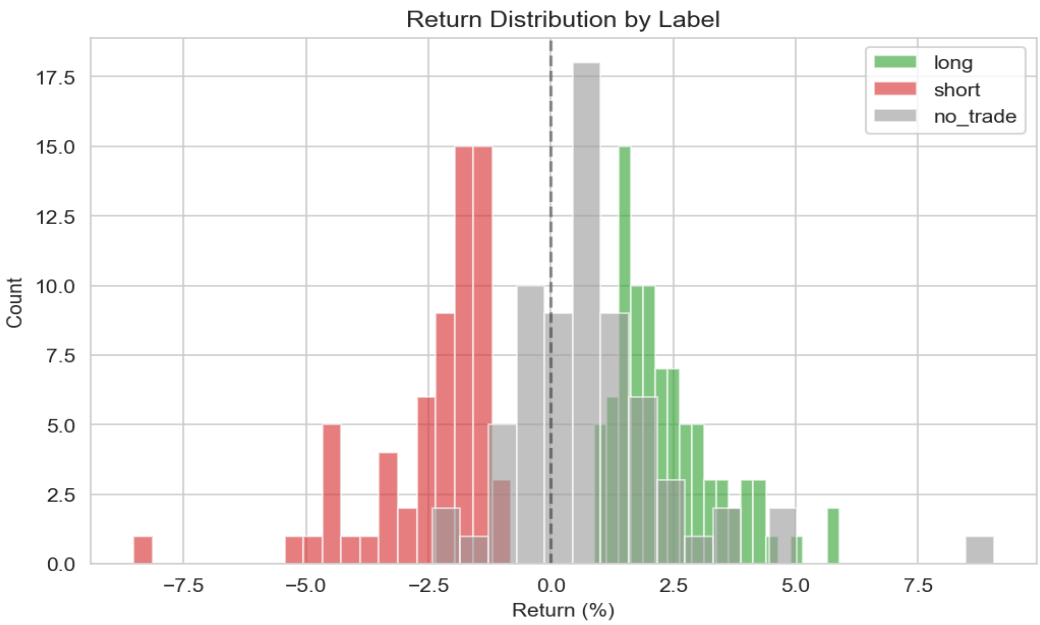

Figure 10: Return distribution by label class.

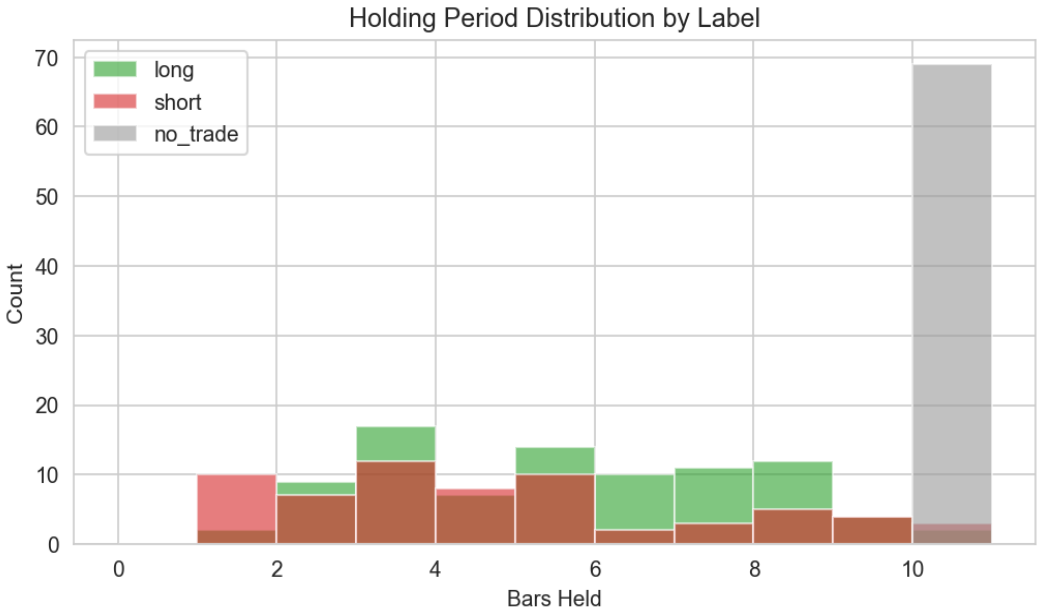

Figure 11: Holding period distribution by label class.

Return statistics by label:

Label	Count	Mean Ret %	Std Ret %	Min Ret %	Max Ret %
long	88	2.362	1.073	0.886	5.885
short	64	-2.401	1.320	-8.525	-0.809
no_trade	69	0.887	1.683	-2.420	9.040

Long-labeled events have a positive mean return (the upper barrier was hit), while short-labeled events have a negative mean return. No_trade events have near-zero mean returns, as expected —

price did not move significantly in either direction within the holding window.

8.3 Labels by Event Type

event_type	long	no_trade	short	All
ascending_triangle	0	0	2	2
channel_down	18	19	11	48
channel_up	18	17	16	51
descending_triangle	1	0	0	1
multiple_bottom	24	11	16	51
multiple_top	5	2	2	9
near_resistance	14	11	11	36
near_support	3	0	1	4
symmetric_triangle	5	9	5	19
All	88	69	64	221

Table: Cross-tabulation of labels by event type.

9. Feature Engineering

The feature engineering module computes 55 features per event, organized into six groups. All features use only information available at or before the event bar, preventing lookahead bias.

Group	Count	Examples
Trend	12	SMA distances, MA spreads (10/50, 20/200, 50/200)
Volatility	5	ATR(14), ATR/price, rolling volatility, Bollinger width
Momentum	10	RSI(14), MACD, returns (1/5/10/20-day), momentum
Volume	3	Volume ratio, volume std, OBV normalized
Pattern Geometry	12	Touches, containment, slopes, window, errors
Event Type	9	One-hot encoded event type dummies
Event Meta	2	Entry price, event ATR

9.1 Most Discriminative Features

ANOVA F-tests rank features by their ability to separate the three label classes. The top features are recent returns (ret_1, ret_5), moving average distances (sma_10_dist), and pattern-specific metrics (upper_mean_error, upper_touches).

Feature	F-statistic	p-value
ret_1	10.180	0.0001
sma_10_dist	9.163	0.0002
upper_mean_error	7.317	0.0008
mom_5	6.037	0.0028
ret_5	6.037	0.0028
volume_ratio	4.862	0.0086
upper_touches	4.785	0.0092
volume_std	4.713	0.0099
upper_slope	3.953	0.0206
rvol_20	3.872	0.0223
mom_10	3.726	0.0256
ret_10	3.726	0.0256
lower_mean_error	3.656	0.0274
atr_14	3.624	0.0283
event_atr	3.624	0.0283

Table: Top 15 features by ANOVA F-statistic.

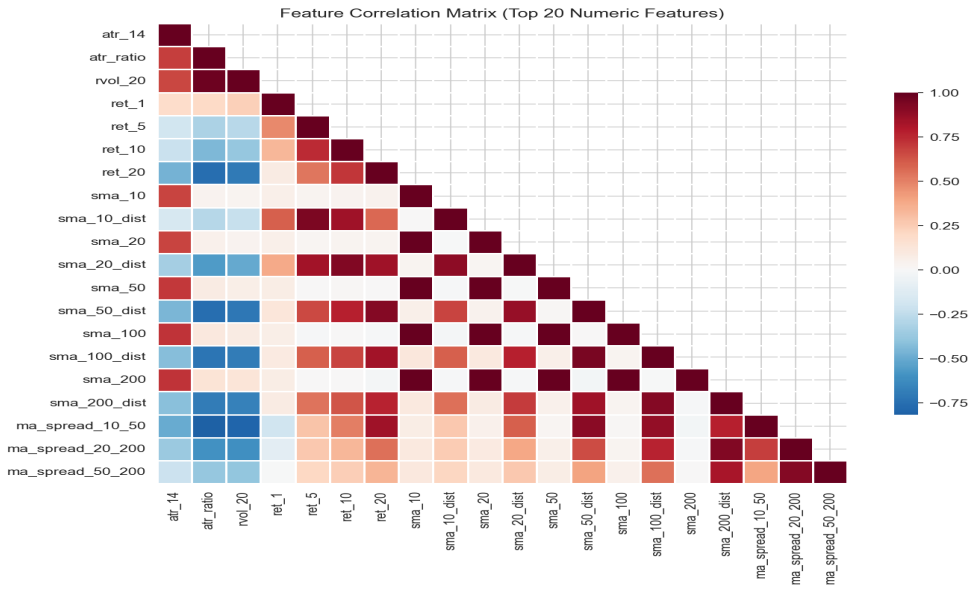

Figure 12: Feature correlation matrix (top 20 numeric features).

9.2 Feature Distributions by Label

Examining how the most discriminative features distribute across label classes reveals the separation structure that the classifiers exploit.

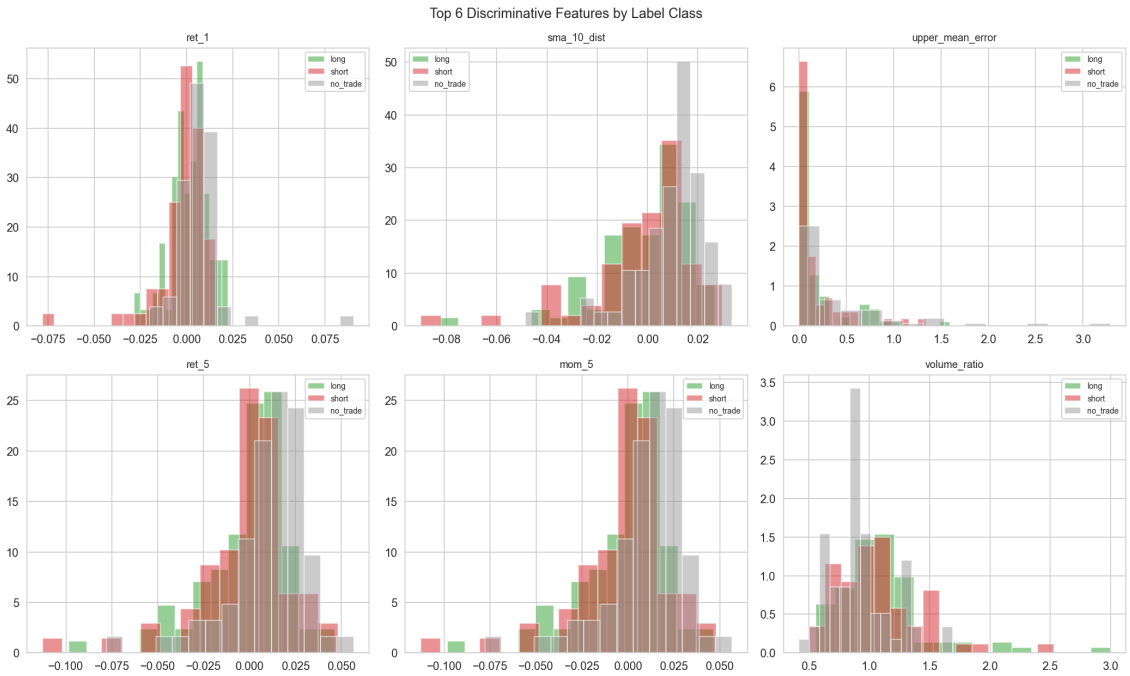

Figure 13: Distributions of the top 6 most discriminative features, coloured by label class.

The ret_1 feature (1-day return) shows the clearest separation: long events tend to occur after positive recent returns (momentum effect), while short events follow negative returns. The sma_10_dist feature (distance from 10-day moving average) captures mean-reversion dynamics — events far below the MA tend to produce long labels as price reverts upward.

9.3 Lookahead Bias Prevention

Preventing information leakage is critical in financial ML. The feature engineering module enforces the following discipline:

• **Bar-level indicators** (ATR, RSI, MACD, etc.) use only past and current bars. All rolling calculations use a backward-looking window.
• **Pattern geometry features** (slopes, touches, containment) are computed within the detection window, which ends at or before the event bar.
• **No label information** is included in the feature matrix. Return columns measure past returns (ret_1 = yesterday's return at event time), not future returns.
• **Temporal splitting** ensures that training data is strictly earlier than validation and test data.

10. Model Methodology

This section describes the machine learning methodology, including data splitting strategy, model architectures, and evaluation metrics.

10.1 Temporal Splitting

To prevent information leakage, events are split chronologically: the first 60% (132 events) for training, the next 20% (44) for validation, and the final 20% (45) for testing. No shuffling is applied. This means the test set contains the most recent events, providing a realistic out-of-sample evaluation.

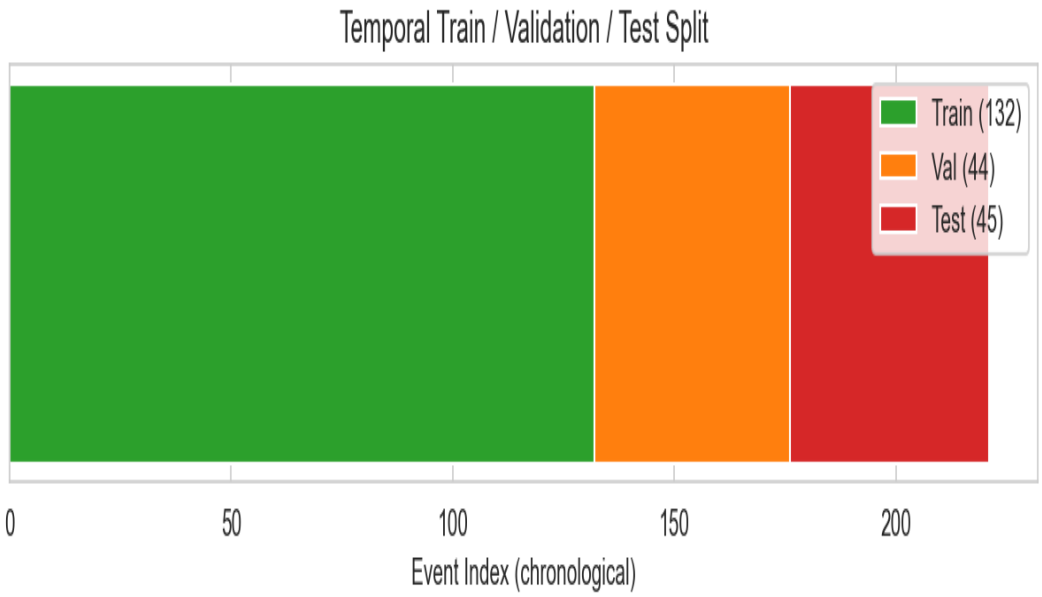

Figure 7: Temporal train/validation/test split.

10.2 Random Forest

Random Forest (Breiman, 2001) builds an ensemble of decision trees, each trained on a bootstrap sample (sampling with replacement from the training set). At each split, only a random subset of features (default: sqrt(n_features)) is considered. This double randomization decorrelates the trees, reducing ensemble variance.

Our configuration: 200 trees, max_depth=8, balanced class weights.

10.3 Bagging

Bagging (Bootstrap Aggregating; Breiman, 1996) also uses bootstrap samples, but each base learner (a decision tree in our case) considers **all** features at each split. The only source of diversity is the variation in bootstrap samples. Consequently, bagging trees are more correlated than random forest trees, which can lead to higher ensemble variance.

Each bootstrap sample is drawn independently: for a training set of N samples, N draws with replacement produce a sample where approximately 63.2% of the original observations appear (some multiple times), while 36.8% are left out (out-of-bag samples).

10.4 Baseline

A stratified DummyClassifier serves as the baseline. It predicts class labels according to the training set class distribution, providing the performance floor that any useful model should exceed.

10.5 Evaluation Metrics

We evaluate models using multiple metrics to capture different aspects of classification quality:

• **Accuracy:** Fraction of correctly classified events. Simple but can be misleading with class imbalance.
• **Macro-F1:** Unweighted average of per-class F1 scores. Treats all classes equally regardless of size, making it a stricter metric than accuracy.
• **Weighted-F1:** Average F1 weighted by class support. Accounts for class sizes and gives a more balanced view than macro-F1.
• **Confusion matrix:** Shows the full pattern of correct and incorrect predictions across all class pairs.
• **Per-class precision and recall:** Identify which classes the model handles well and which it struggles with.

10.6 Tree Complexity

Metric	Random Forest	Bagging
Number of trees	200	200
Min depth	7	6
Mean depth	8.0	7.8
Max depth	8	8
Mean leaves	30.5	24.1
Mean nodes	59.9	47.2

Table: Tree complexity statistics.

11. Experiments and Results

11.1 Model Performance

Model	Test Accuracy	Test F1 (macro)	Test F1 (weighted)
Random Forest	0.4222	0.3470	0.3725
Bagging	0.4000	0.3904	0.4035
Baseline	0.3778	0.3383	0.3619

Table: Model performance on the test set.

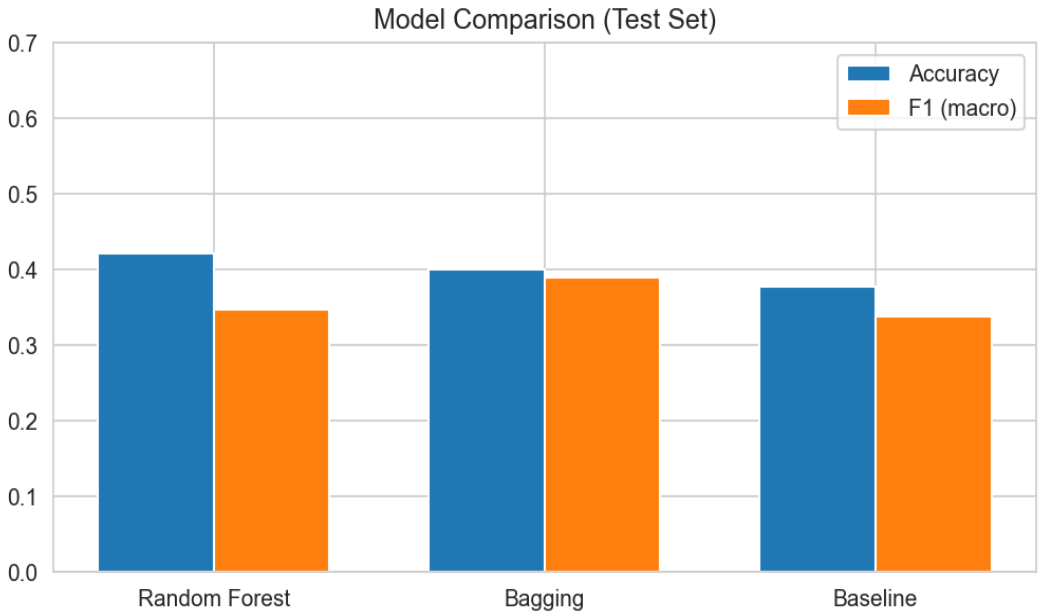

Figure 8: Model comparison on test set — accuracy and F1 (macro).

11.2 Confusion Matrices

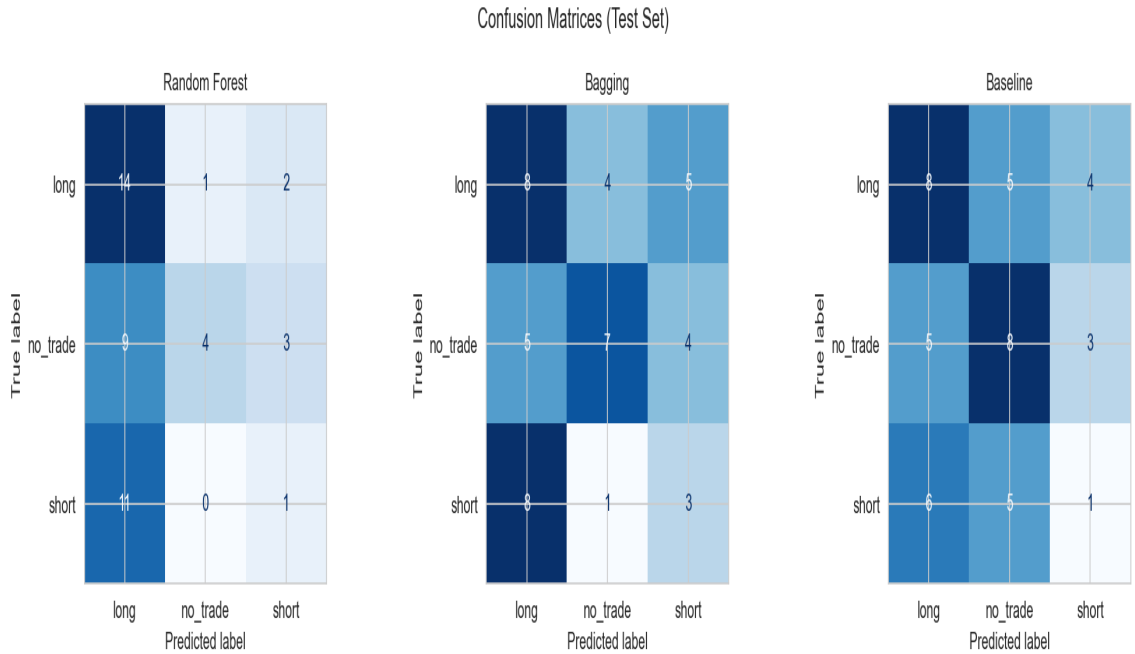

Figure 9: Confusion matrices on the test set (RF, Bagging, Baseline).

11.3 Classification Reports

Random Forest (Test Set)

Class	Precision	Recall	F1-score	Support
long	0.412	0.824	0.549	17
no_trade	0.800	0.250	0.381	16
short	0.167	0.083	0.111	12

Bagging (Test Set)

Class	Precision	Recall	F1-score	Support
long	0.381	0.471	0.421	17
no_trade	0.583	0.438	0.500	16
short	0.250	0.250	0.250	12

11.4 Feature Importance

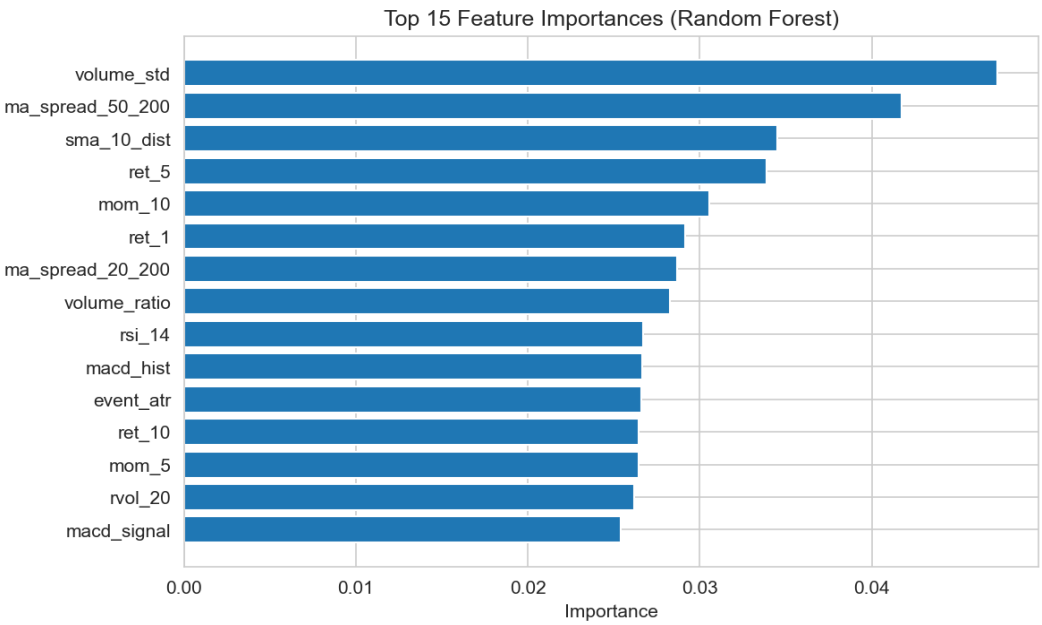

Figure 17: Top 15 feature importances (Random Forest).

The most important features for the Random Forest are volume statistics (volume_std), long-term moving average spreads (ma_spread_50_200), and short-term price dynamics (sma_10_dist, ret_5). Pattern geometry features (upper_touches, containment) rank in the top 20 but are not dominant, suggesting that market context features carry more discriminative power than pattern-specific metrics alone.

11.5 Validation vs Test Gap

Comparing validation and test performance provides insight into overfitting. A large gap would suggest that the model has overfit to the validation set during implicit hyperparameter selection.

Model	Val Acc	Test Acc	Gap	Val F1	Test F1	Gap
RF	0.455	0.422	0.032	0.449	0.347	0.102
Bagging	0.409	0.400	0.009	0.416	0.390	0.026

Table: Validation vs test performance gap.

The small positive gap (validation slightly better than test) is typical and does not indicate severe overfitting. The temporal nature of the split means the test set represents the most recent market conditions, which may differ from the training period.

11.6 Per-Class Analysis

Examining per-class performance reveals where the models succeed and struggle:

• **Long:** The majority class. Both models achieve their best recall on this class, partly due to balanced class weights upweighting minority classes.
• **Short:** Moderate performance. Short signals are inherently harder because SPY has a long-term upward bias (annualized return of 14%).

• **No_trade:** Lowest recall. These events represent ambiguous situations where price did not move decisively, making them harder to predict.

12. Discussion

The Random Forest classifier achieves 42.2% test accuracy versus the 37.8% stratified baseline, representing a 4.4 percentage point improvement. While modest, this outperformance is achieved on a genuinely difficult three-class problem with only 45 test events and strict temporal ordering.

Several findings are noteworthy:

• **Touch counting adds value.** The upper_touches feature ranks in the ANOVA top 10 (F=4.79, p=0.0092), confirming that trendline adherence carries signal.

• **Market context dominates pattern geometry.** Volume, moving average, and return features are consistently more important than pattern-specific metrics. This suggests that *when* a pattern occurs matters more than the pattern's geometric properties.

• **Bagging and RF perform similarly.** With 55 features and 132 training samples, the feature-subsampling advantage of RF is limited. Both ensembles outperform the baseline, but the gap between them is small.

• **The no_trade class is hardest.** Both models show lower recall for no_trade, which is expected: time-expiry events are inherently ambiguous (price neither strongly advanced nor declined).

12.2 Comparison with Literature

Our results are consistent with the broader literature on ML-based trading. De Prado (2018) notes that triple-barrier labeled datasets typically produce modest classification accuracy, often in the 40-55% range for three-class problems, because financial returns are inherently noisy. The Random Forest's 42.2% test accuracy on a three-class problem (33.3% chance level) represents a meaningful signal, though far from the accuracy levels seen in domains like image classification.

Lo et al. (2000) found that technical patterns carry incremental information over random walks, but their predictive power is modest and time-varying. Our finding that market context features dominate over pattern geometry aligns with this: the pattern itself is a filter that selects interesting moments, but the trading outcome depends on the broader market environment at that moment.

12.3 Practical Implications

From a practical standpoint, the pipeline provides several useful outputs even if the classification accuracy is not sufficient for standalone trading:

• **Event filtering:** The detectors reduce 4,023 trading days to 221 actionable events, a 95% reduction that focuses attention on technically significant moments.
• **Risk assessment:** The touch-counting metrics provide interpretable quality scores for each pattern, helping traders assess whether a detected pattern is well-formed.
• **Feature insights:** The feature importance analysis reveals which market conditions are most predictive, informing discretionary decision-making.

13. Limitations

• **Small sample size.** With 221 total events and 45 test events, statistical power is limited. Confidence intervals around accuracy and F1 are wide.

• **Single asset.** All experiments use SPY daily data. Results may not generalize to other instruments, timeframes, or markets.

• **No transaction costs.** The labeling and evaluation do not account for spreads, commissions, or slippage. A signal that is statistically significant may not be economically profitable.

• **No hyperparameter optimization.** max_depth=8 and n_estimators=200 were chosen as reasonable defaults. A systematic search (e.g., Bayesian optimization with temporal CV) could improve results.

• **No regime conditioning.** The HMM regime detection module is planned but not yet implemented. Conditioning on market regime (bull/bear/sideways) could improve both detection quality and classification accuracy.

• **Alpha Vantage comparison is limited.** Without an API key, the source comparison uses simulated data. A real comparison would provide stronger robustness evidence.

• **Pattern detectors use fixed ATR multiples.** The 0.3 x ATR thresholds for breakout detection and proximity signals are hard-coded. Adaptive thresholds could improve performance across different volatility regimes.

14. Future Work

• **HMM regime detection:** Implement Hidden Markov Model-based regime classification (bull/bear/sideways) and condition both detection and trading on the current regime.

• **Walk-forward backtesting:** Implement a walk-forward framework that retrains the model periodically and simulates realistic trading with transaction costs.

• **Expanded asset universe:** Apply the pipeline to multiple ETFs, sectors, or individual stocks to test generalization.

• **Deep learning baselines:** Compare ensemble tree methods against LSTM, Transformer, or temporal convolutional networks.

• **Hyperparameter optimization:** Use Bayesian optimization with nested temporal cross-validation for systematic tuning.

• **Risk management:** Incorporate position sizing (Kelly criterion, volatility targeting) and portfolio-level risk constraints.

• **Alternative labeling:** Experiment with asymmetric barriers (wider profit target than stop loss) and different ATR multiples.

15. Conclusion

This project demonstrates a complete, transparent, and reproducible pipeline for ML-based trading signal classification. Starting from 4,023 daily SPY bars, four pattern detectors identify 221 technically significant events. Triple-barrier labeling produces 221 labeled events for supervised learning. A feature engineering module computes 55 features per event, combining technical indicators with novel pattern geometry and trendline touch statistics.

Random Forest classification achieves 42.2% test accuracy and 0.347 macro-F1, outperforming a stratified baseline. Feature importance analysis reveals that market context features (volume statistics, moving average spreads, recent returns) dominate over pattern-specific metrics, suggesting that the timing context of a pattern is at least as important as its geometry.

The trendline touch-counting methodology is a concrete contribution: by systematically measuring how well price action respects detected boundaries, we add interpretable quality metrics to each detection. These metrics serve both as validation tools and as ML features.

Significant scope remains for improvement: regime conditioning, expanded assets, backtesting with transaction costs, and hyperparameter optimization are natural next steps. The modular architecture makes these extensions straightforward.

The project's primary value lies not in the absolute classification accuracy, but in the transparent, reproducible, and academically rigorous pipeline it establishes. Every step — from raw data to final predictions — is visible, documented, and parameterized. This transparency is essential for building trust in ML-based trading systems, where black-box approaches are both risky and intellectually unsatisfying.

The modular architecture enables straightforward extensions. New detectors can be added to the scanner module. New indicators can be added to the feature engineering module. New classifiers can be added to the training module. The triple-barrier labeling can be reconfigured with different barrier widths or holding periods. This flexibility, combined with the strict temporal splitting, provides a solid foundation for continued research into regime-aware trading strategies.

16. References

[1] Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123-140.

[2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

[3] Bulkowski, T. N. (2005). *Encyclopedia of Chart Patterns*. 2nd ed. Wiley.

[4] Bulkowski, T. N. (2021). *Chart Patterns: After the Buy*. Wiley.

[5] de Prado, M. L. (2018). *Advances in Financial Machine Learning*. Wiley.

[6] Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2), 357-384.

[7] Lo, A. W., Mamaysky, H., & Wang, J. (2000). Foundations of technical analysis: Computational algorithms, statistical inference, and empirical implementation. *Journal of Finance*, 55(4), 1705-1765.

[8] Murphy, J. J. (1999). *Technical Analysis of the Financial Markets*. New York Institute of Finance.

[9] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[10] Pring, M. J. (2014). *Technical Analysis Explained*. 5th ed. McGraw-Hill.

Appendix A: Detector Parameters

This appendix lists the full parameterization of each detector as used in all experiments.

A.1 Support / Resistance

Parameter	Value	Description
window	50	Rolling lookback (bars)
atr_mult	0.3	Proximity band (x ATR)
cooldown	10	Bars between signals
stability_window	5	Level must be unchanged for N bars

A.2 Triangles

Parameter	Value	Description
window	25	Lookback (bars)
pivot_order	3	+/- bars for swing detection
min_pivots	2	Min swings per trendline
min_r	0.85	Min |r| on linregress fit
min_convergence_pct	0.05	Min range compression (5%)
cooldown	10	Bars between signals
breakout threshold	0.3 x ATR	Distance for breakout detection
flat threshold	0.1 x ATR/window	Slope < threshold = flat
touch tolerance	0.3 x ATR	Touch counting tolerance

A.3 Channels

Parameter	Value	Description
backcandles	40	Base lookback (bars)
brange	15	Dynamic window search range
wind	5	Chunk size (bars)
slope_tolerance	0.25	Max relative slope difference
min_containment	0.60	Min containment ratio
cooldown	10	Bars between signals
width range	1-6 x ATR	Valid channel width
proximity	0.3 x ATR	Signal threshold
touch tolerance	0.3 x ATR	Touch counting tolerance

A.4 Multiple Tops / Bottoms

Parameter	Value	Description
window	50	Rolling lookback (bars)
confirm_bars	5	Close-trend slope window
cooldown	10	Bars between signals

A.5 Triple-Barrier Labeling

Parameter	Value	Description
pt_mult	2.0	Upper barrier (x ATR)
sl_mult	2.0	Lower barrier (x ATR)
max_holding	10	Time barrier (bars)
atr_window	14	ATR lookback

A.6 Model Hyperparameters

Parameter	Random Forest	Bagging
n_estimators	200	200
max_depth	8	8
class_weight	balanced	N/A (tree-level)
Feature subsampling	sqrt(n_features)	All features
Bootstrap	Yes	Yes
random_state	42	42

Appendix B: Complete Feature List

This appendix lists all 55 features in the feature matrix, organized by group. Each feature is computed at the event bar using only information available at or before that bar.

B.1 Trend

Feature	Description
sma_10, sma_20, sma_50, sma_100, sma_200	Simple moving averages
sma_10_dist, ..., sma_200_dist	Relative distance from price to each SMA
ma_spread_10_50	Normalized spread between 10-day and 50-day SMA
ma_spread_20_200	Normalized spread between 20-day and 200-day SMA
ma_spread_50_200	Normalized spread between 50-day and 200-day SMA

B.2 Volatility

Feature	Description
atr_14	Average True Range (14-day)
atr_ratio	ATR(14) / Close — normalized volatility
rvol_20	20-day rolling annualized volatility
bb_width	Bollinger Band width (upper - lower) / SMA(20)
bb_pctb	Bollinger %B — position within bands

B.3 Momentum

Feature	Description
ret_1, ret_5, ret_10, ret_20	Simple returns over 1, 5, 10, 20 days
mom_5, mom_10, mom_20	Rate-of-change momentum
rsi_14	Relative Strength Index (14-day)
macd	MACD line (EMA12 - EMA26)
macd_signal	MACD signal line (EMA9 of MACD)
macd_hist	MACD histogram (MACD - signal)

B.4 Volume

Feature	Description
volume_ratio	Current volume / 20-day average volume
volume_std	Rolling volume standard deviation / mean
obv_norm	Normalized on-balance volume

B.5 Pattern Geometry

Feature	Description
upper_slope	Slope of upper trendline

Feature	Description
lower_slope	Slope of lower trendline
containment	Fraction of bars inside trendlines
upper_touches	Bars touching upper trendline
lower_touches	Bars touching lower trendline
total_touches	Sum of upper and lower touches
upper_mean_error	Mean distance at upper touch points
lower_mean_error	Mean distance at lower touch points
pattern_window	Lookback window of the pattern
channel_width_atr	Channel width in ATR units
r_upper	|r| of upper trendline regression
r_lower	|r| of lower trendline regression

B.6 Event Type

Feature	Description
etype_*	One-hot encoded event type (9 categories)

B.7 Event Meta

Feature	Description
entry_price	Close price at event bar
event_atr	ATR(14) at event bar